# C3 decomposition of the incumbent's counted S at fp28

# C3 per-phase decomposition of counted cost, v6 counted session (fp28)

Local, zero-cost. No AWS, no EKS, no commits. Observations only.

Input: `/tmp/claude-0/-home-user/fe537fb0-7f2e-504d-8df2-39e551cfffea/scratchpad/v6/counted-untrusted/session/records.jsonl` sha256 `1a7f3faad6f5ebada35f4546d150e0f35aadf0fae3df9d84234832851b8ec00b` (matches 1a7f3faa…). 576 records = 4 arms x (16 warmup + 128 measured). Only the 128 measured per arm are used (16 targets = 2 curves x 8; 8 rounds each). Read with `python3 -I`; scripts in `scripts/`.

Method: cost is the record's `cost.total_gae` (w = 0, i.e. counted group ops only) and `cost.s` = total_gae / sqrt(r). Each phase is the record's `phases[].gae`. Rounds are averaged within a target; share = sum(phase)/sum(total) over targets; 95% CI is a percentile bootstrap over the 16 targets (10,000 resamples, seed 20261106). Phase S = S x phase share. Residual = total_gae minus the sum of phase gae.

## 1. Per-phase shares (w = 0)

Incumbent (`walk` targets, `arm=incumbent`): mean S 20.771, mean total 307,472 GAE/run (290,184 on the first curve, 324,760 on the second).

| Phase | Share of S | 95% CI | Mean S |
|---|---|---|---|
| oracle_setup | 54.007 % | [52.45, 55.64] | 11.259 S [10.89, 11.63] |
| relations (walk trials + restarts + jump table) | 45.981 % | [44.35, 47.54] | 9.509 S [9.14, 9.86] |
| verify | 0.013 % | [0.012, 0.013] | 0.0026 S |
| factor_base | 0 | | 0 GAE |
| linear_algebra | 0 | | 0 GAE (row ops are unpriced at w=0) |
| **Residual (total - sum of phases)** | **0.000 %** | [0, 0] | 0 |

The residual is exactly zero: `total_gae = oracle_setup + relations + verify` in every record. Candidate arm (layout=lines) gives the same shares to 4 digits.

rho-neg: mean S 0.9032; search 97.114 % [96.76, 97.41]; setup 2.595 % [2.33, 2.91]; internal_verification 0.290 % [0.257, 0.330]; residual 0. rho-neg-aa is the same to 3 digits.

### Incumbent relations phase, split further (derived from native counters)

| Item | Share of S | 95% CI | How obtained |
|---|---|---|---|
| walk steps (additions) | 45.071 % | [43.42, 46.65] | `walk_steps` counter (exact) |
| restart additions | 0.0135 % | [0.0128, 0.0141] | `walk_pool_restarts` (exact) |
| restart offset draws (`walk_pool_ops`: lazy, scalar mults + add) | 0.396 % | [0.387, 0.405] | counter (exact) |
| jump table (16 jumps = 32 scalar mults + 16 adds) + initial point (2 mults + 1 add) | 0.500 % | [0.488, 0.513] | **derived**: relations adds+doubles minus the three lines above |

| Item | Share of S | 95% CI | How obtained |
|---|---|---|---|
| unexplained inside relations | 0 | | |

The derived line is a reconstruction, not a counter: no record field isolates jump-table set-up. Its ~1537 ops over 34 scalar mults is about 45 ops/mult against about 38 ops/mult inside `walk_pool_ops`; I did not resolve that gap.

**RANKING.md section-2 style check (S - set-up - trials)**

Using trials = `targets_tried` at 1 GAE each: mean set-up 11.259 S, trials 9.322 S, mean S 20.771, remainder **0.189 S (0.91 %), CI [0.184, 0.195] S**, not the ~1.5 S in RANKING.md. Per curve: r=191,494,577 (seed …101) S 20.970, remainder 0.200 S; r=249,209,281 (seed …102) S 20.572, remainder 0.179 S. RANKING.md used r=249,209,281 for both curves and a `lookups_uncharged`-based trial count; the two curves in the session have different r, and total_gae differs by curve (mean 290,184 vs 324,760).

## 2. Fields present / missing

Present per record: `phases[]` (name, adds, doubles, scalar_mults, gae, wall_ns, native, work_native) with phases factor_base, oracle_setup, relations, linear_algebra, verify for IC and setup, search, internal_verification for rho; `work.phases[]` (bit-op prims per phase); `counters` (trials, relations, matrix); rho `counters` (setup_additions, setup_doublings, start_additions, look_ahead_additions, walk_operations, verification_operations …). Relations-phase natives: trials, walk_steps, walk_restarts, walk_pool_restarts, walk_pool_offsets, walk_pool_ops, walk_jumps, target_guard_probes, lookups. Missing: any separate counter for jump-table scalar mults or the initial-point scalar mults (they are folded into the relations adds/doubles; `scalar_mults`=66 is a count only, its ops are inside adds/doubles), any renewal counter (the incumbent has none; renewals appear only for stride-renew targets), and a separate verification cost inside relations (verify is its own phase, 39 GAE/run, 1 scalar mult).

## 3. Is the repeated-base fixed-window generator the default at ea8e9ef7?

No. Reading `src/cryptanalysis/ic_boundary.rs` at ea8e9ef7 (extracted to `src/ic_boundary.rs` here; line numbers are the file's): - `CountedGroup::mul` is plain double-and-add (ic_boundary.rs:1132-1147); no impl overrides it (`impl CountedGroup for PrimeCurve` at 4246, BinaryGroup 4732; the only `fn mul(` in the file are 1133 and an unrelated modular 4222). - Walk jump-table draw: `draw_jumps` calls `g.mul` for `generator` and `target` separately (ic_boundary.rs:7866-7869, then `g.add`). - Walk first segment: two `g.mul` calls plus an add (ic_boundary.rs:8494-8496). - Restarts are not scalar multiplications: each restart after the first adds one pooled offset (`g.add`, 8516-8521), the offset drawn lazily by `draw_pool_offset` (7882-7893, via `draw_jumps` with two `g.mul`). - Restart pool default is Lazy (`restart_pool()`, 9163-9170). - The only joint aP+bQ routine is `joint_mul` (1410-1440, Shamir joint double-and-add, not fixed-window, not repeated-base), used by stride targets (2471-2486, 2689-2695, 2987-2991), not by `Targets::Walk`. - Pathspec-limited `git grep` for `fixed.window|repeated.base|92c130|window_table|comb_table` over ic_boundary.rs, ic_framework/ and ecbench/ at ea8e9ef7 returned nothing. Not searched: other `src/` directories (no repo-wide grep per instructions).

Files: `decomp.json`, `phase_shares.svg`, `phase_shares.png`, `scripts/decomp.py`, `scripts/explore.py`, `src/` (extracted sources).

# Section 2. oracle_setup breakdown (incumbent `walk`)

Script: `scripts/oracle_setup.py`; full numbers in `decomp.json` key `section_2_oracle_setup` (and `oracle_setup.json`). 128 measured records, 16 targets.

## 2.1 Counters inside `oracle_setup` (mean per run, bootstrap 95% CI over targets)

| Counter | Mean | 95% CI | min..max over 128 runs |
|---|---|---|---|
| adds | 166,056 | [166056, 166056] | 166056..166056 |
| doubles | 0 | [0, 0] | 0..0 |
| scalar_mults | 0 | [0, 0] | 0..0 |
| gae | 166,056 | [166056, 166056] | same |
| table_insert_attempts | 165,649 | [165649, 165649] | same |
| lookups (x_index reads) | 407 | [407, 407] | same |
| point negations in this phase | 0 | | (407 negations are in `factor_base`, `index_point_negations`) |
| wall_ns (diagnostic only) | 6,049,025 | [6,037,432, 6,060,876] | |

All counters are identical in all 128 runs (same N=407, deterministic), so the CIs are degenerate. Table entries: no record field carries `PairTable.entries`; only insert attempts exist. 166,056 - 165,649 = 407 sums were identity and skipped.

## 2.2 What `oracle_setup` builds (ea8e9ef7, `src/cryptanalysis/ic_boundary.rs`)

`PairTable::build_negation_folded`, lines 6370-6408 (doc comment 6363-6369). Loop at 6381-6393: for each of the A=407 abscissae, i = first signed point of that abscissa (6382), and j over every signed point (2 per abscissa) of abscissae ai..end (6383-6384); one `g.add(fb.points[i], fb.points[j])` per pair (line 6385), identity skipped (6386-6388), insert of `key>>1` (6389), `attempts += 1` (6390). - Number of additions = sum_ai 2(407-ai) = 407*408 = N(N+1) = 166,056. Matches the counters exactly. Scales as N^2, not N. - It is NOT N scalar multiplications. scalar_mults = 0, doubles = 0. No scalar, no bit length. - Operands are the factor-base points themselves (the 814 signed points of the 407 smallest abscissae, from `prime-abscissa`), not multiples of a fixed (P, Q); they vary per pair. Neither G nor the target Q is involved. - The dispatch from the `mitm:negation_folded=1` oracle option to this function was not traced in this pass; the match with the counters (adds exactly N(N+1), attempts = adds - 407) is the evidence.

## 2.3 Per-scalar-multiplication cost implied by counters

Undefined: scalar_mults = 0 in this phase. The only ratio is 166,056 additions / 407 = 408 additions per factor-base abscissa (N+1). For reference, textbook double-and-add for a 28-bit scalar: 27 doublings + 14 expected additions = 41 ops (the incumbent's measured scalar mults elsewhere were about 38-45 ops each, from section 1).

## 2.4 Formula-only alternatives (MODELED, not measured)

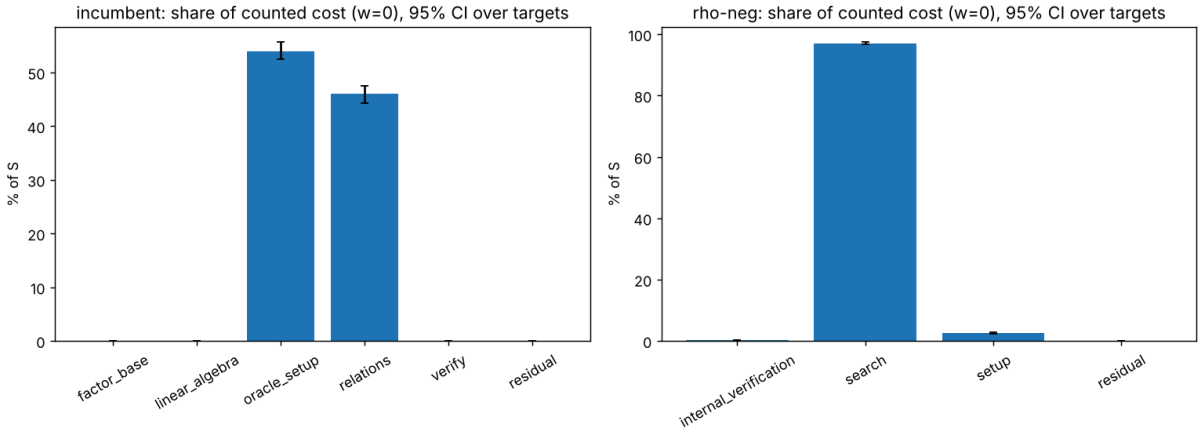
Premise caveat: since the measured phase has no scalar multiplications, these rows are the cost of a hypothetical in which N=407 scalar mults of one fixed base (or 407 joint aP+bQ) are done, with L=28-bit scalars and 1 GAE per add or double. They are not alternatives to the N(N+1)-addition pair table. Measured oracle_setup: 166,056 GAE.

| Method | Precompute (GAE) | Per mult (GAE) | Total per run, N=407 (GAE) |
|---|---|---|---|
| textbook double-and-add (one fixed base) | 0 | 41.00 | 16,687 |
| fixed-base comb w=4 (h=7) | 32 | 12.56 | 5,145 |
| fixed-base comb w=5 (h=6) | 50 | 10.81 | 4,451 |
| fixed-base comb w=6 (h=5) | 82 | 8.92 | 3,713 |
| fixed-base comb w=7 (h=4) | 144 | 6.97 | 2,980 |
| fixed-base comb w=8 (h=4) | 275 | 6.98 | 3,118 |
| fixed-base windowed, no doublings, w=4 (h=7 tables) | 122 | 5.56 | 2,386 |

| Method | Precompute (GAE) | Per mult (GAE) | Total per run, N=407 (GAE) |
|---|---|---|---|
| fixed-base windowed, no doublings, w=5 (h=6 tables) | 205 | 4.81 | 2,164 |
| fixed-base windowed, no doublings, w=6 (h=5 tables) | 334 | 3.92 | 1,930 |
| fixed-base windowed, no doublings, w=7 (h=4 tables) | 525 | 2.97 | 1,733 |
| fixed-base windowed, no doublings, w=8 (h=4 tables) | 1040 | 2.98 | 2,255 |
| Shamir joint aP+bQ (a,b both 28-bit), P+Q precomputed | 1 | 48.00 | 19,537 |

Formulas: textbook = (L-1) + L/2. Comb (Lim-Lee, single table, h=ceil(L/w)): precompute (2^w-1-w) + (w-1)h; per mult (h-1) + h(1-2^-w). Windowed without doublings (tables of j*2^(w i)P): precompute h(2^w-2) + w(h-1); per mult max(0, h(1-2^-w)-1). Shamir: (L-1) + 0.75L, plus 1 add for P+Q. Optimistic assumptions: expected digit-zero skips, no inversion/affine cost differences between add and double, 1 GAE per op.

## Figure: phase shares of S (w = 0), from phase_shares.png




Phase shares of the incumbent's counted S at w = 0 (v6 counted records)

*This figure section is appended by the PDF render; DECOMP.md itself is unchanged (sha256 885579d5…).*